ECEN 468/719 Advanced Logic Design

Department of Electrical and Computer Engineering

Texas A&M University

Lab 7: Design of SRAM and Synthesis

Objectives

In this lab, we will implement a 256K x 8-bit SRAM design using Verilog. We will use Synopsys VCS to simulate the designed model for verification. We will also use Synopsys Design Analyzer to synthesize the design.

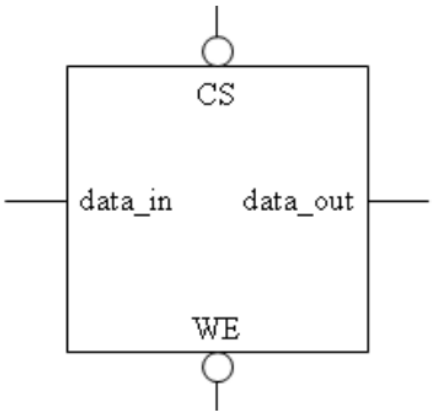
A Brief Introduction to SRAM

We will implement a Static Random Access Memory (SRAM) with a storage cell structure that does not require a refresh. Therefore, it operates faster than Dynamic Random Access Memory and is used as fast-cache memory in a computer.

We will start with implementing a simple SRAM cell. Then we will do (1)functional simulation, (2)synthesis, and (3)gate simulation with netlist on the simple SRAM cell. After that, we will implement a 256K SRAM and only do a functional simulation. The 256K SRAM will be used in future labs.

The block diagram of a basic SRAM cell is shown in the Figure below. It has active-low inputs for both cell select (CS_b) and write enable (WE_b). The cell select signal is generated by a decoder that selects among multiple cells in the same system. Note the absence of a clock

signal. Storage registers and register files are implemented by flip-flops, while the storage devices of SRAM or DRAM are implemented as transparent latches. This implementation supports asynchronous operations and minimizes the time a RAM requires service from a shared bus.



Implementation & Simulation for a Single SRAM Gate

Please login to the Olympus server and create a working directory for this lab using the following commands.

```
## Create and navigate to the working directory.
mkdir -p $HOME/ECEN468/Lab7/src1
cd $HOME/ECEN468/Lab7/src1
```

Download the zip file (lab7_code_a.tar.gz) from the lab web page and extract it. In the extracted folders, you will find five files named **"SRAMCELL.v", "SRAMCELL_tb.v", "generic.sdb", "osu018_stdcells.db",** and **"osu018_stdcells.v"**. Copy them to the working directory.

You will implement your code in **"SRAMCELL.v".** Please also review **"SRAMCELL_tb.v"** to understand the test bench.

Requirements:

1. SRAM is sensitive to CS and WE.
2. Output high impedance when CS is high.
3. WE = 1: Read from SRAM; WE = 0: Write to SRAM.

Once you complete the implementation, please verify the correctness of your design. We will first compile the design using the commands below.

```
load-ecen-468 (skip this line on machines in Zach 127)
source /opt/coe/synopsys/vcs/W-2024.09-SP2-4/setup.vcs.sh
vcs -full64 SRAMCELL_tb.v
```

If it shows compile errors, please recheck your design and fix your code. If compilation is complete, go to the next step to show the results.

```
./simv
```

Please take screenshots of the terminal output and include them in the report. Please justify the correctness of your design by interpreting the output.

To view the graphical waveform of the simulation results, run the following command to invoke WaveView, and open "wave.dump" in WaveView

```
source /opt/coe/synopsys/wv/V-2023.12-4/setup.wv.sh
wv &
```

The graphical waveform is for debugging. You don't need to include it in the report.

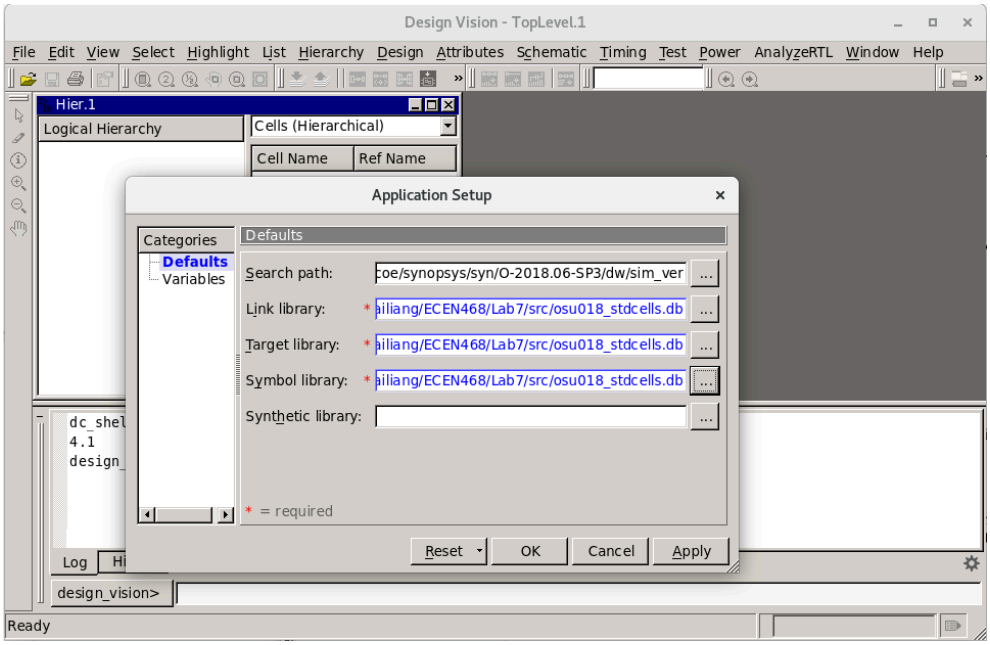
Once the verification is complete, we will use **Design Vision** to synthesize the design. For your convenience, please invoke Design Vision from a separate directory to isolate the intermediate files generated.

```
## Create and navigate to a separate working directory.
mkdir $HOME/ECEN468/Lab7/src1/dv_Work
cd $HOME/ECEN468/Lab7/src1/dv_Work
## Launch Design Vision
source /opt/coe/synopsys/syn/V-2023.12-SP1/setup.syn.sh
design_vision &
```

In Design Vision, first we want to link the libraries.

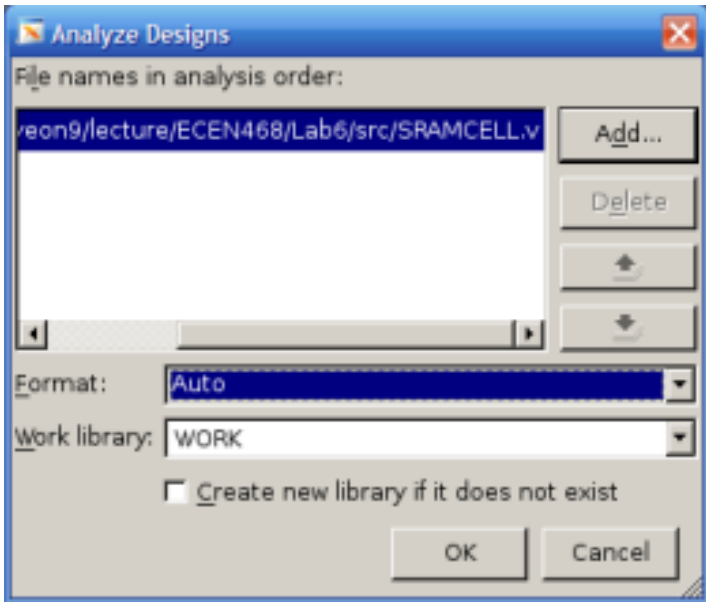
- Click on File -> Setup -> Defaults
- Add osu018_stdcells.db to "Link Library" (Remove all other library files)
- Add osu018_stdcells.db to "Target Library" (Remove all other library files)

- Add osu018_stdcells.db to “Symbol Library” (Remove all other library files)
- Click on OK



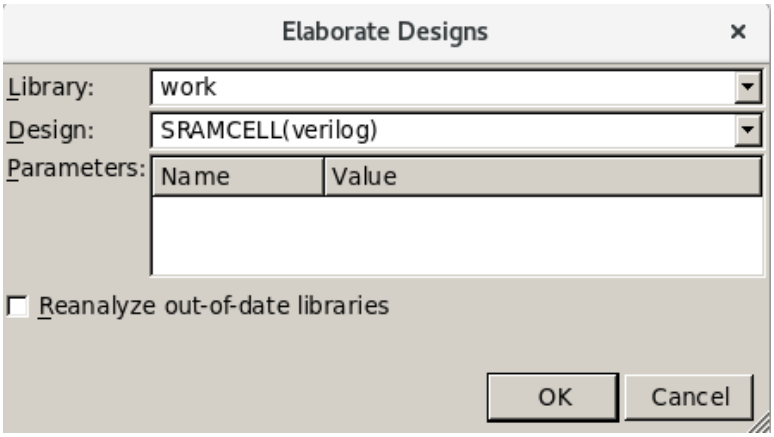
Now we want to *Analyze* the design. In *Analyze*, it reads VHDL or Verilog files, checks for proper syntax and synthesizable logic, and stores the design in some intermediate format.

- Click on File -> Analyze.
- in the pop-up window, add “SRAMCELL.v”, and click on OK as shown in the figure below.



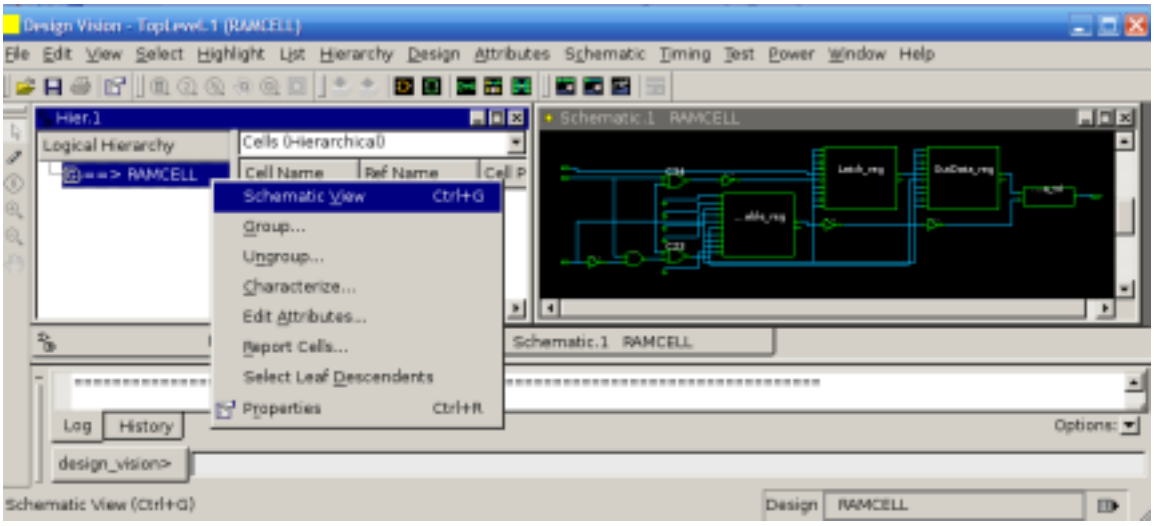
Now we want to *Elaborate* the design. In *Elaborate*, it creates a design from the intermediate format produced by *Analyze*. It replaces the HDL operators with synthetic operators and determines the correct bus sizes.

- Click on File -> Elaborate.
- In the pop-up window, type “work” in *Library*, and select SRAMCELL in *Design*, as shown in Figure X.
- Click on OK.



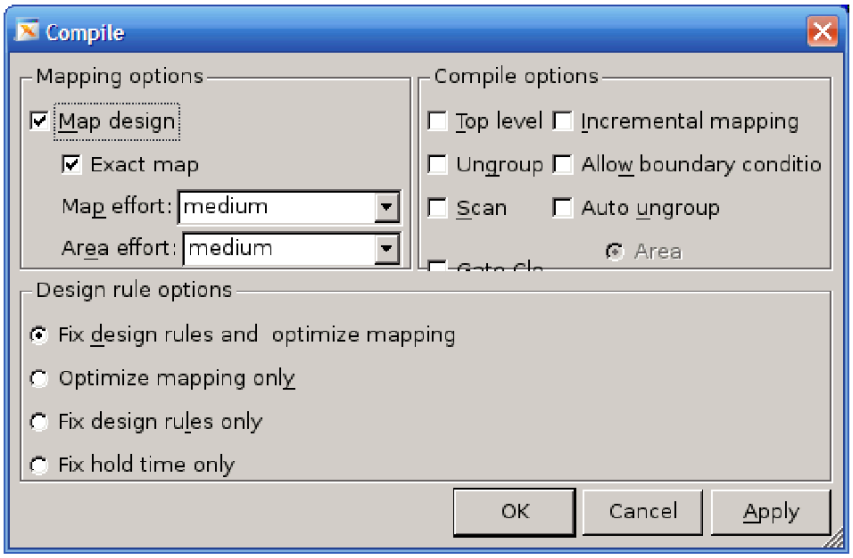
Print Schematic:

- Right-click on SRAMCELL -> Schematic View, as shown in Figure X.
- You can double-click on the SRAMCELL block in the schematic to check the detail of the netlist.
- Click on File -> Print to save the schematic as a PDF file.



Synthesize the module:

- Click on Design -> Compile Design.
- In the pop-up window, click on OK as shown in Figure X.



After synthesis, we want to save the optimized netlist as a Verilog file:

- File -> Save as
- Please enter "SRAMCELL_gate.v" as the file name and choose "Verilog" as the file type.
- Check the option "Save All Designs in Hierarchy".

Save the Standard Delay Format (SDF) by running the following command in the command window:

```
write_sdf SRAMCELL.sdf
```

You may now close Design Vision.

With the SRAM cell saved as Verilog and SDF, we will do a Gate Simulation on it. First, copy these two files out of the current directory:

```
cp ./SRAMCELL_gate.v $HOME/ECEN468/Lab7/src1/
cp ./SRAMCELL.sdf $HOME/ECEN468/Lab7/src1/
cd $HOME/ECEN468/Lab7/src1/
```

We will use the previous testbench as a template to create a testbench for the gate simulation.

```
cp SRAMCELL_tb.v SRAMCELL_gate_tb.v
```

Please open the new testbench "SRAMCELL_gate_tb.v" and do the following modifications:

- Make sure the following three lines are on the top of the file.
  ```
  `timescale 1ns/10ps
  `include "SRAMCELL_gate.v"
  `include "osu018_stdcells.v"
  ```
- Remove the following line if it exists.
  ```
  `include "SRAMCELL.v"
  ```
- Make sure the following two lines are at the bottom of the file. (within the module entity)
  ```
  initial
        $sdf_annotate("SRAMCELL.sdf", SRAMCELL_01);
  ```
- Change the name of the dump file in the following line in the test bench.
  ```
  $dumpfile ("wave_gate.dump");
  ```

Please make sure you have all the files listed below available in your current directory.
`SRAMCELL_gate.v, SRAMCELL.sdf, osu018_stdcells.v, SRAMCELL_gate_tb.v`

Once we have the files ready, we can run the following command to start the simulation.

```
source /opt/coe/synopsys/vcs/W-2024.09-SP2-4/setup.vcs.sh
vcs -full64 SRAMCELL_gate_tb.v
```

If no error occurs, do the next step.

```
./simv
```

Please take a screenshot of the simulation output, and include it in the report.

Implementation & Simulation for an SRAM Array

Now we proceed to implement the SRAM array. We want to create another directory for it.

```
## Create and navigate to the working directory.
mkdir -p $HOME/ECEN468/Lab7/src2
cd $HOME/ECEN468/Lab7/src2
```

Copy SRAM.v and SRAM_tb.v from lab7_code_b.tar.gz to this directory.

Please follow the procedure listed in the previous section, implement the SRAM Array design in SRAM.v, and use the testbench defined in SRAM_tb.v for simulation.

Submission

Please only submit one PDF file containing the following items:

SRAM Cell:

1. Screenshots of the simulation output after running ./simv.
2. Justification of the correctness of the results.
3. Screenshots of the code “SRAM_gate.v”.

SRAM Array:

1. Screenshots of the simulation output after running ./simv.
2. Justification of the correctness of the results.
3. Screenshots of the code “SRAM.v”.